LEV: Thinking Fast and Deep with Looped Decision Models

Dingjie Song[1,*], **Zhiling Yan**[1,*], **Yuxuan Zhang**[2,3], **Lichao Sun**[1,†]

[1]Lehigh University, [2]University of British Columbia, [3]Vector Institute
[*]Equal contribution, [†]Corresponding author

Decision models answer typed questions about an input with one probability per option, computed in a single forward pass without generating text. Like the fast System 1 of dual-process theory, they answer at once, but their depth is fixed at training time, so they spend the same computation on easy and hard questions. Models that reason in generated text can think longer, but they pay for every extra step in decoded tokens. We present LEV, a decision model that thinks deeper by rerunning its layers in a loop instead of generating tokens. LEV builds on a language model pretrained as a loop, reads the options after every loop step and trains all steps jointly, so one checkpoint can answer at any depth. On three looped backbones, accuracy rises with depth on the same weights, mostly on hard questions, and a 2.6B LEV goes from 63.2% at depth one to 76.2% at depth four on JevBench. On-demand depth, set per question by a small stopper with a threshold fitted on unlabelled requests, saves 15% to 24% of the computation at a cost of under one accuracy point, level with or slightly above a fixed mix of depths. On a locked test set, the 1.4B and 3B models are 5.4 to 8.8 points more accurate than dense (non-looped) models and cascades of the same latency below 50 ms.

Code: https://github.com/OpenLAIR/lev
Models: https://huggingface.co/collections/songdj/lev
Website: https://openlair.github.io/lev/

1 Introduction

Generative language models that reason with chain-of-thought act as a System 2 (Kahneman, 2011). They write out their reasoning before choosing an option, so every decision is slow, even an easy one (Villalón, 2026; razorback16, 2026), and turning the reasoning off lowers their accuracy markedly on JevBench (Standhartinger, 2026). Decision models such as Jev (TypeSafe AI, 2026) and Kev (Palmer, 2026) are a System 1 for questions with a fixed set of options. They return a probability for each option from one forward pass (Devlin et al., 2019; Robinson et al., 2023) in tens of milliseconds, but they spend the same computation on every question and cannot think longer on a hard one. Not every question needs more computation. On the public JevBench items, Kev-4B answers 28 more items correctly than Kev-0.8B, and Figure 2a shows that all of them come from the standard and hard tiers, because both models already answer every easy item. A decision model that could add computation only where it pays off would keep the speed of the small model on easy questions and approach the accuracy of the large one on hard questions.

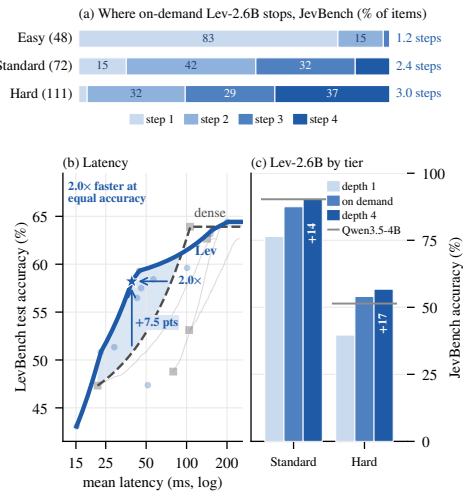


Figure 1 (a) Loop steps that on-demand LEV-2.6B spends per JevBench tier. (b) LevBench test accuracy against latency, with the Pareto frontiers of LEV over all depths (solid) and of dense models and cascades (dashed). Arrows mark the gap of on-demand LEV-3B to the dense frontier. (c) JevBench accuracy of LEV-2.6B on the standard and hard tiers at depth one, on demand, and at depth four, against the dense Qwen3.5-4B control.

Existing ways to spend more computation without generating text fall short for decision models (Section 3). A cascade passes uncertain questions to a larger model (Varshney &

Baral, 2022; Chen et al., 2024), but reads them again and needs a second model. Early exit stops at a confident intermediate layer (Xin et al., 2020; Schuster et al., 2022; Elhoushi et al., 2024), but these layers are not trained to be read and are often confidently wrong on new data (Meronen et al., 2024). A loop reruns the same layers without adding parameters (Dehghani et al., 2019; Geiping et al., 2025), but added to a pretrained model it barely helps, as Jev-LCT (Cao, 2026) reports, unless it gets far more continued pretraining than a decision model is trained on (McLeish et al., 2025; Koishekenov et al., 2025).

We propose Lev, a System 1.5 decision model (Wang et al., 2025) that thinks in loops rather than tokens. Lev builds on natively looped language models (Zhu et al., 2025; Nanbeige Lab, 2026), whose loop is trained in pretraining rather than added afterwards. Decision training therefore does not have to teach the model to use its loop, whereas a loop added to a pretrained decision model stays unused (Section 3). A shared readout scores the options after each step and all steps are trained jointly, so one set of weights answers at any depth, and a further step resumes from the previous hidden states without reading the input again. A lightweight stopper sets the depth of each question on demand by predicting whether its answer already agrees with the deepest one, with a threshold set on unlabelled requests. We use this target because the usual early-exit rule, a confidence threshold set on labelled data, does not beat a fixed mix of depths here, as shallow readouts are overconfident on new data. Since the target is the model's own deepest answer, a deployment can fit the stopper on its own traffic without labels.

We build Lev on three looped backbones and evaluate it on JevBench and two held-out benchmarks. Accuracy rises with depth on the same weights, mostly on hard questions (Figure 1c), and the stopper spends more steps on harder questions without seeing their difficulty (Figure 1a). On-demand depth saves 15% to 24% of the computation at a cost of under one accuracy point, level with or slightly above a fixed mix of depths. A locked test, read once after the recipe and the stopper were fixed, confirms that on-demand depth stays within one point of the deepest readout. Below 50 ms, the 1.4B and 3B models are 5.4 to 8.8 points more accurate than dense (non-looped) models and cascades of the same latency (Figure 1b).

Our contributions are as follows.

- We propose Lev, a System 1.5 decision model that thinks deeper on hard questions through loop steps rather than generated text. It reads the options after every step of a natively looped backbone, so one checkpoint serves every depth.
- We show that a stopper whose threshold is set on unlabelled requests gives on-demand depth without labels. It predicts agreement with the deepest answer rather than correctness, and it is level with or above a fixed mix of depths where confidence thresholds set on labelled data fall below it.
- On three looped backbones, Lev gains most from depth on hard questions, and below 50 ms it is more accurate than dense models and cascades of the same latency. We also build LevBench, a benchmark frozen before any model read it, whose tiers move step by step away from the training data.

2 Related Work

Decision models. Decision models score options discriminatively in one forward pass (Devlin et al., 2019; Zaratiana et al., 2024; Robinson et al., 2023; Zheng et al., 2024), and block attention masks let independent questions share that pass (Ratner et al., 2023). Jev and its open reimplementation Kev read the options with a pointer head (Vinyals et al., 2015). A generative model can also answer such questions by naming an option, but it must bind each option to its symbol (Robinson et al., 2023) and is sensitive to the order and labels of the options (Zheng et al., 2024). Open Jev-style servers such as djev (Villalón, 2026) and OpenJev (razorback16, 2026) follow this generative route and can add reasoning text before the answer, which raises accuracy on hard questions at the price of decoding time on every question. A decision model instead scores the options themselves in one pass, and its probabilities can be calibrated with a single temperature (Guo et al., 2017). Lev adds per-question depth to such a model through a backbone pretrained as a loop.

Looped models and adaptive computation. Looped transformers rerun one block of layers to add computation without adding parameters (Dehghani et al., 2019; Giannou et al., 2023), and looped language models such as Ouro (Zhu et al., 2025) reason better with more iterations and can be read at several depths

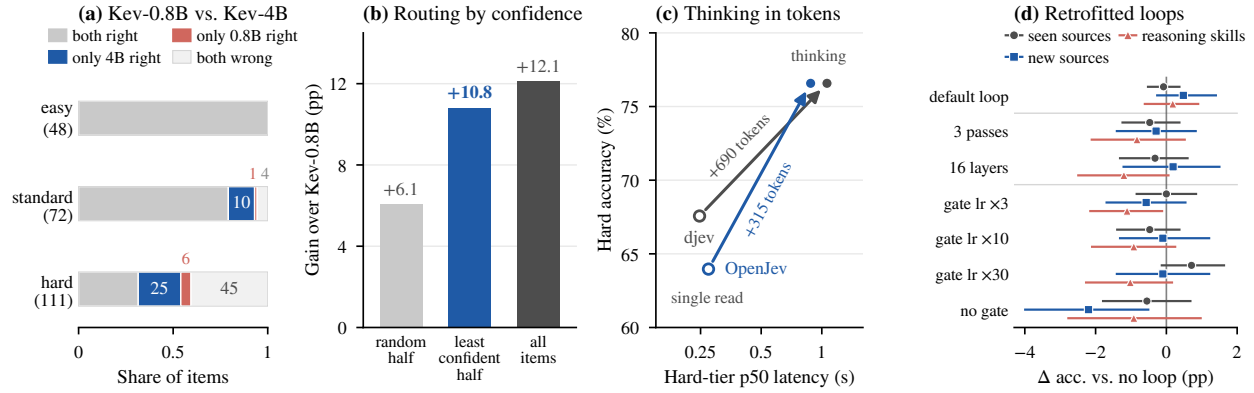


Figure 2 Evidence for depth chosen per question. (a) Outcome of each public JevBench item under Kev-0.8B and Kev-4B. (b) Gain over Kev-0.8B when Kev-4B answers a random half, the least confident half, or all items. (c) Single read against generated reasoning on the same base model, with latencies from published results on different endpoints. (d) Accuracy change from loops retrofitted onto Qwen3.5-0.8B, each row changing one setting of the default.

(Geiping et al., 2025; Saunshi et al., 2025; Jeddi et al., 2026; Popescu et al., 2026). A loop added to a standard pretrained model needs adapters or continued pretraining (Bae et al., 2025; McLeish et al., 2025) and otherwise tends to stay unused (Chen et al., 2026; Shapiro, 2026). Adaptive computation stops early through halting (Graves, 2016) or confident exits (Xin et al., 2020; Elbayad et al., 2020; Schwartz et al., 2020), or adds latent tokens (Hao et al., 2025; Goyal et al., 2024), and its shallow exits are often overconfident (Meronen et al., 2024) unless calibrated toward the full model, as in CALM (Schuster et al., 2022). CATs (Schuster et al., 2021) likewise predicts whether an exit agrees with the full model, and BERxiT (Xin et al., 2021) learns when to exit from labelled data. LEV instead trains every loop step as a decision model, stops whole steps without adding positions, and sets the threshold of its stopper on unlabelled requests.

3 Motivation

A decision model spends the same computation on every question. To test whether this is a good use of computation, we compare the released Kev-0.8B and Kev-4B, trained on the same sources, item by item on the 231 public JevBench items (Section A), and we try three ways of adding computation. They lead to two observations.

Observation 1: Extra computation pays off on few questions, and a cheap first read finds them. (1) **The gain comes from hard questions.** Kev-4B answers 28 more items correctly than Kev-0.8B, and Figure 2a shows that this gain comes entirely from the standard and hard tiers, because both models already answer every easy item. (2) **The small model can tell which questions need more.** The two models reach different outcomes on only 42 of the 231 items, yet Kev-4B spends its extra parameters on every item. The confidence of Kev-0.8B finds these items. As Figure 2b shows, sending its least confident half to Kev-4B nearly matches Kev-4B on all items.

Observation 2: Existing ways to add computation cost too much or stay unused. (1) **Cascades read uncertain questions twice.** Kev-0.8B already takes 79 ms at the median against 108 ms for Kev-4B, so every item the cascade of Figure 2b passes on takes longer than Kev-4B alone, is read again from the start, and needs a second model to host. (2) **Thinking in tokens pays on every question.** JevBench scores two open Jev-style servers, djev (Villalón, 2026) and OpenJev (razorback16, 2026), with and without generated reasoning on the same DiffusionGemma 26B base model. These results in Figure 2c show that thinking raises hard accuracy by 9 to 13 points but makes the median hard item three to four times slower, and every question pays for its reasoning text, including easy ones. (3) **Retrofitted loops stay closed.** We loop eight middle layers of Qwen3.5-0.8B-Base, the backbone of Kev-0.8B, behind per-channel gates that start at zero, and train the model with Kev's recipe (Section A.3). Figure 2d shows that the loop changes accuracy by less

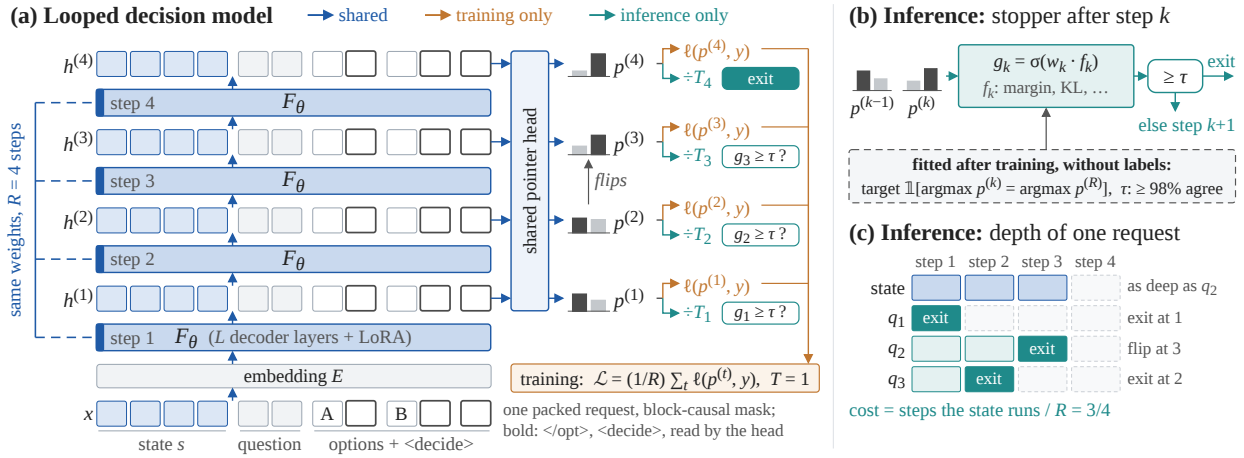


Figure 3 LEV with R loop steps, drawn for $R = 4$. (a) A request is packed into one sequence, and the same decoder stack F_θ runs R times. One shared pointer head reads the `</opt>` and `<decide>` positions after every step. (b) After step k, a stopper predicts whether the answer already agrees with the deepest one. (c) Each question exits at its own depth, and the state runs as deep as its deepest question.

than half a point, and that a higher gate learning rate, a third pass, or a wider block does not help. The gates stay near zero, so the loop stays closed, while removing the gates lowers accuracy on new sources by 2.2 points. A backbone pretrained as a loop is open from the start.

4 Method

4.1 Preliminaries

Decision models. A request holds a state s and questions $q_1, \ldots, q_m$, and question q_j lists options $o_{j1}, \ldots, o_{jK_j}$. Following Kev, the state comes first, and each question follows with its options wrapped in delimiter tokens and a final `<decide>` token. A block-causal mask lets each question attend only to the state and to its own tokens, and its position indices restart after the state, so each question is answered as if it were asked alone while the state is encoded once. With h_{dec} the final hidden state at `<decide>` and h_k the one at the closing delimiter of option k, the pointer head computes

$$z_k = \frac{(W_K h_k)^\top (W_Q h_{\text{dec}})}{\sqrt{d_p}}, \tag{1}$$

where $W_Q, W_K \in \mathbb{R}^{d_p \times d}$ are learned, and returns $p = \text{softmax}(z/T)$ with a temperature T fitted after training. Training minimises log loss $\ell(p, y) = -\log p_y$ at $T = 1$, or cross-entropy for the few questions whose label y is a distribution.

Looped transformers. A looped transformer applies one stack of L decoder layers F_θ several times with the same weights (Dehghani et al., 2019; Saunshi et al., 2025). From the token embeddings $h^{(0)} = E(x)$, step t computes

$$h^{(t)} = \text{Norm}\big(F_\theta(h^{(t-1)})\big), \quad t = 1, \ldots, R, \tag{2}$$

where Norm is the final normalisation layer, so R steps reach an effective depth of $L \cdot R$ layers with the parameters of L (Geiping et al., 2025). Ouro ($R = 4$, 7.7T pretraining tokens) applies its language-modelling head and next-token loss after every step, so every step ends in a state that the backbone is trained to read.

4.2 Looped backbone with a readout at every step

LEV keeps the decision model of Section 4.1 and replaces its backbone with a natively looped language model, Ouro or Nanbeige4.2 (Section 5). Figure 3a shows the resulting model, in which the same stack runs R times

and one pointer head reads the options after every step. Applying the pointer head of Equation (1) to the hidden states $h^{(t)}$ of every step gives the logits $z^{(t)}$ and one readout $p^{(t)} = \mathrm{softmax}(z^{(t)}/T_t)$ per question and depth, whose answer we write $\hat{o}^{(t)} = \arg\max p^{(t)}$. The head and the LoRA adapter are shared across steps, and training averages the loss over steps at $T_t = 1$,

$$\mathcal{L} = \frac{1}{R}\sum_{t=1}^{R} \ell\big(p^{(t)}, y\big), \tag{3}$$

so every depth is trained to be a decision model. Step t depends only on earlier steps, so serving at depth k computes only the first k steps at about k/R of the full cost, and one pass to depth R yields all R readouts, which we use for evaluation.

Shallow and deep readouts can be miscalibrated by different amounts, so we fit one temperature T_k per depth by minimising the log loss on the calibration partition of the training suite (Guo et al., 2017). No benchmark is used for fitting, and since temperature scaling keeps the predicted option, accuracy at each depth does not depend on T_k.

4.3 Deciding when to go deeper

LEV reads every question at depth one, and after each step a stopper decides which questions still need another one (Figure 3b). A further step continues from the hidden states of the last one, so the request is never read again. A question can only go deeper together with its state, so, as Figure 3c shows, the state s of a request runs to depth $d(s) = \max_j d(q_j)$.

What the stopper predicts. A confidence threshold chosen for accuracy on labelled data transfers poorly for decision models, because shallow readouts are overconfident on new data and blind to the errors that more steps would fix (Section 7.3). At each depth $k < R$, the stopper instead predicts whether the current answer already agrees with the deepest one, the consistency target of CATs,

$$y_k = \mathbf{1}\big[\hat{o}^{(k)} = \hat{o}^{(R)}\big], \quad \hat{y}_k = \sigma\big(w_k^\top \phi_k + b_k\big), \tag{4}$$

a logistic regression per depth. The features ϕ_k come from the readouts at depths k and $k-1$ only: the top probability, the margin between the two most likely options, the normalised entropy, whether the answer equals that at depth $k-1$, $\mathrm{KL}(p^{(k-1)} \,\|\, p^{(k)})$, the change of the top probability, $\log K$, and k. With a threshold τ, question q stops at

$$d_\tau(q) = \min\big(\{k < R : \hat{y}_k(q) \geq \tau\} \cup \{R\}\big). \tag{5}$$

Fitting without labels. The target y_k is the model's own deepest answer, so fitting the stopper needs no correct answers, and neither does the threshold. We take the smallest τ, and so the cheapest stops, under which at least 98% of the final answers $\hat{o}^{(d_\tau(q))}$ agree with the deepest answers $\hat{o}^{(R)}$ on the fitting questions. A deployment can therefore fit the stopper on the calibration partition together with unlabelled requests from its own traffic, which we emulate with two folds of each test panel split by record group (Section E).

Cost. The cost of a request is the depth $d(s)$ of its state divided by R, and we average it over requests weighted by their length in tokens. A fixed depth k costs k/R, and serving a share of the requests at depth k and the rest at depth $k+1$ reaches any cost in between, with the accuracies mixed in the same shares. We call this piecewise-linear curve the *fixed-depth frontier*, and on-demand depth is useful only if it is more accurate than the frontier at the same cost.

5 Experimental Setup

Models. We build LEV on three looped backbones. Ouro-1.4B and Ouro-2.6B run their stack $R = 4$ times and were pretrained with a loss after every step. Nanbeige4.2-3B runs its stack twice and was pretrained with a loss on its last step only, so it tests whether LEV needs intermediate steps that were supervised in pretraining. We call the three models LEV-1.4B, LEV-2.6B, and LEV-3B. The dense controls are Qwen3-1.7B-Base (Yang

System	Depth	Latency ↓	Tuning ↑			Held-out ↑			Benchmark ↑
		(ms)	Kev-Seen	Kev-Skills	Kev-Transfer	Kev-Breadth	LevBench	Avg.	JevBench
LEV (one checkpoint per model, read at every depth)									
LEV-1.4B Ouro-1.4B	1	**15**	78.7	47.6	50.9	57.0	44.5	50.8	58.0
	2	30	83.7	58.2	60.4	63.4	53.0	58.2	65.4
	3	44	85.7	62.5	65.9	66.4	56.4	61.4	68.0
	4	58	85.9	63.7	66.4	66.0	58.1	62.1	68.0
LEV-2.6B Ouro-2.6B	1	51	79.0	51.9	56.6	55.7	46.9	51.3	63.2
	2	101	85.7	65.9	66.1	65.3	58.6	62.0	71.0
	3	150	<u>86.2</u>	70.4	71.3	67.1	<u>62.6</u>	<u>64.8</u>	<u>75.3</u>
	4	200	86.1	<u>71.4</u>	**72.5**	**67.7**	**63.6**	**65.7**	**76.2**
LEV-3B Nanbeige4.2-3B	1	23	82.6	59.6	62.0	62.6	51.8	57.2	69.3
	2	45	86.0	66.9	70.0	66.1	59.4	62.8	72.7
Dense controls, same data and recipe									
Qwen3-1.7B	–	<u>22</u>	81.3	50.8	58.2	58.3	48.0	53.2	63.6
Qwen3.5-2B	–	105	83.3	63.3	62.8	64.7	53.1	58.9	68.8
Qwen3.5-4B	–	140	**87.9**	**72.9**	<u>71.4</u>	<u>67.4</u>	62.1	64.7	73.6
Released references, trained differently									
Kev-0.8B	–	79	83.4	56.0	57.8	58.9	47.4	53.2	63.6
Kev-4B	–	108	88.1	69.2	73.3	67.7	63.3	65.5	75.8
Jev 1.13.0	–	652†	83.5	77.7	80.7	76.0	69.2	72.6	86.6

Table 1 Accuracy (%) and median latency per request of every system on the six panels of Table 5. Avg. is the unweighted mean of Kev-Breadth and LevBench. Among the models trained on identical data, the LEV models and the dense controls, bold marks the best value and underline the second best. †Measured through the production API with the network included, and not comparable to the local measurements.

et al., 2025), Qwen3.5-2B-Base, and Qwen3.5-4B-Base. As references we report the released Kev-0.8B and Kev-4B and the closed Jev 1.13.0.

Training. Every model is trained with LoRA (Hu et al., 2022) of rank 16 for one epoch on the same 31,329 records, which merge the training sources of Kev-0.8B into a single stage. No label comes from Jev, so LEV is not distilled from the model it is compared with. Section B gives the settings and the data. Training uses up to four H200 GPUs per model.

Evaluation panels. We fixed the recipe and chose the design of the stopper on three tuning panels from frozen Kev suites, Kev-Seen (training sources), Kev-Transfer (new sources), and Kev-Skills (held-out skill templates). Our models read the two held-out panels only after that. LevBench is a suite we build and froze before any model read it. Its tiers move step by step away from the training data, from new items of the training sources to datasets never trained on, and Section B.4 describes its construction. Kev-Breadth holds 14 datasets from five application areas that Kev never trains on. We also report the 231 public JevBench items, of which 48 are easy, 72 standard, and 111 hard. Table 5 lists all panels.

Metrics. We report accuracy on every panel and the expected calibration error (ECE) on the hard tier of JevBench, which we score with JevBench's own code (Section B.5). For on-demand depth we also report the cost of Section 4.3. Latency is the median per request on one H200 GPU in bf16. Confidence intervals are 95% paired bootstrap intervals that resample groups of related questions.

6 Main Results

Table 1 supports four conclusions. (1) Extra loop steps raise accuracy on the same weights, with diminishing returns. The gain holds for every backbone and on every panel and reaches 13.0 points on JevBench for LEV-2.6B (Figure 13), while the fourth step of the Ouro models adds at most one point. Most of the gain comes with the first extra step, which lifts the held-out average of LEV-2.6B from 51.3% to 62.0%, against 65.7% at depth four. It is also smallest on the training sources, as LEV-1.4B and LEV-2.6B gain about 7 points on Kev-Seen from depth one to depth four, against 9.0 to 19.5 points on the four panels further from the training data. (2) One checkpoint covers several operating points and can beat a dense control at the same latency. LEV-3B at depth one is 4.0 points above Qwen3-1.7B on the held-out average at 23 against 22 ms, and LEV-2.6B at depth two is 3.1 points above Qwen3.5-2B at 101 against 105 ms, while Qwen3-1.7B is level with or above LEV-1.4B (Figure 4). (3) LEV matches larger models with fewer parameters but not fewer operations. LEV-3B is level with Qwen3.5-4B on JevBench, and LEV-2.6B with the released Kev-4B (Figure 13), but each loop step runs the whole backbone. (4) On-demand depth saves computation at almost no cost in accuracy and beats dense cascades at low latency (Sections 7.2 and 7.4).

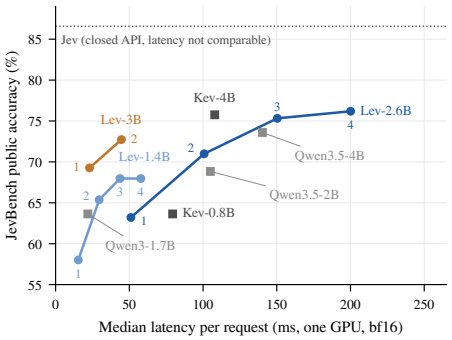


Figure 4 Accuracy and median latency on the 231 public JevBench items. Each LEV checkpoint gives one operating point per depth, labelled by depth. Squares are separate dense checkpoints.

7 Analysis

We analyse LEV through five research questions. RQ1 asks where extra depth helps, by difficulty tier, by revision between steps, and by application area. RQ2 asks whether on-demand depth is more accurate than a fixed mix of depths at the same cost, and RQ3 asks which stopping rule makes it so. RQ4 compares LEV with dense models and cascades at the same measured latency, and RQ5 asks how joint training, more data, and full-weight fine-tuning change the accuracy at each depth.

7.1 RQ1: Where does extra depth help?

Gains concentrate on hard items. From depth one to depth four, LEV-2.6B gains a net 10 of the 72 standard JevBench items and 19 of the 111 hard items, while the easy tier is nearly solved at every depth (Figure 5a). Most revisions are corrections, as 29 hard items become correct and 10 wrong.

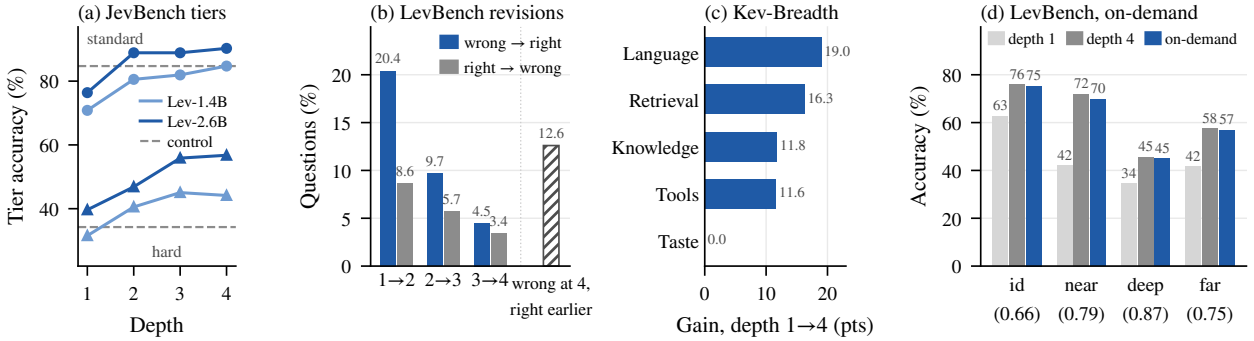


Figure 5 Where depth helps. (a) JevBench accuracy on the standard and hard tiers, with the Qwen3-1.7B control dashed. (b) LevBench questions that LEV-2.6B revises between consecutive depths, or gets right at some shallower depth but wrong at depth four. (c) Gain of LEV-2.6B from depth one to four per Kev-Breadth area. (d) LEV-2.6B on the LevBench tiers, with the cost of on-demand depth in parentheses.

Model	Deepest	On-demand	Cost	vs. deepest	vs. frontier
Development					
LEV-1.4B	61.3	61.0	0.78	−0.3 [−0.6, −0.0]	+0.5 [−0.0, +1.0]
LEV-2.6B	65.3	64.6	0.76	−0.7 [−1.0, −0.4]	+0.2 [−0.4, +0.8]
LEV-3B	62.1	61.3	0.85	−0.8 [−1.1, −0.5]	+1.0 [+0.6, +1.3]
Locked test					
LEV-1.4B	60.1	59.3	0.80	−0.7 [−1.0, −0.5]	+0.5 [+0.1, +0.9]
LEV-2.6B	64.6	63.7	0.78	−0.9 [−1.1, −0.6]	+0.1 [−0.4, +0.5]
LEV-3B	60.7	59.8	0.87	−1.0 [−1.2, −0.7]	+0.9 [+0.6, +1.2]

Table 2 On-demand depth on the two held-out panels, pooled over their questions. Cost is the token-weighted share of the deepest pass. The last two columns are paired differences in accuracy points with 95% CIs. Tables 10 and 11 give every panel.

Depth revises answers, and helps some areas more than others. On LevBench, the first extra step of LEV-2.6B fixes 20.4% of the questions and breaks 8.6% (Figure 5b), and later steps change fewer. For 12.6% of the questions some shallower depth is right while the deepest is wrong, and a stopper that predicts agreement with the deepest answer cannot recover them. Figure 5c shows that on Kev-Breadth depth adds 19.0 points on language understanding but nothing on the two taste tasks, Humicroedit and cfcolor. Extra steps likely add the serial computation that combining facts needs, which taste does not use.

7.2 RQ2: Does on-demand depth beat a fixed depth?

On-demand depth computes less at almost no accuracy cost. Figure 6 shows that on the held-out panels the stopper stays within one point of the deepest readout while computing 76% to 85% of its cost, and Table 2 gives the pooled numbers. The same holds on every panel of Table 10 except the 231 JevBench items.

The gain over the fixed-depth frontier depends on its shape. Against a fixed depth of the same cost, the gain is small. On-demand LEV-2.6B is 1.4 points above the fixed-depth frontier on the tuning panels but only 0.2 points on the held-out panels, while LEV-3B gains 1.0 point there. In Figure 6 the frontier of the Ouro models is concave because the first extra step brings most of the gain, whereas the two-step frontier of LEV-3B is a straight line that most stopping signals beat.

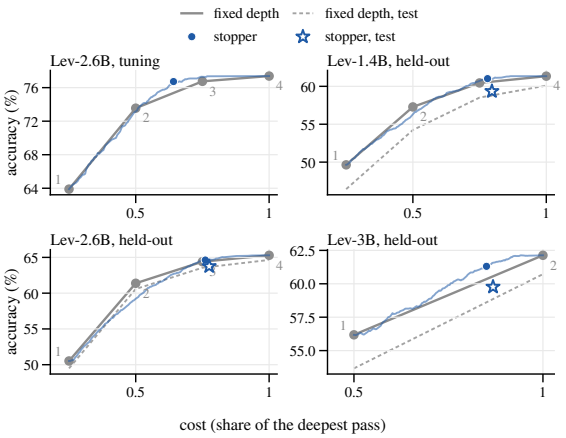


Figure 6 Accuracy against cost. Gray points and line are the fixed depths and their frontier, and the blue line moves the threshold of the stopper to the chosen dot. The dashed line and the star repeat both on the locked test partitions.

The locked test confirms the development result. On the test partitions of both held-out panels, read once after the backbones, the recipe, and the stopper were fixed, on-demand depth again loses at most 1.0 point against the deepest readout, is above the frontier for LEV-1.4B and LEV-3B, and is level with it for LEV-2.6B (stars in Figure 6).

Harder and less familiar items get more steps. The stopper follows difficulty without seeing it. On JevBench, on-demand LEV-2.6B uses 1.2, 2.4, and 3.0 steps on easy, standard, and hard items (Figure 1a). On LevBench it computes 66% of the deepest pass on the *id* tier, 1.8 points above the frontier, and 75% to 87% on the tiers further from the training data (Figure 5d), and the same holds on further panels and on harder generated questions (Table 12 and Figure 7).

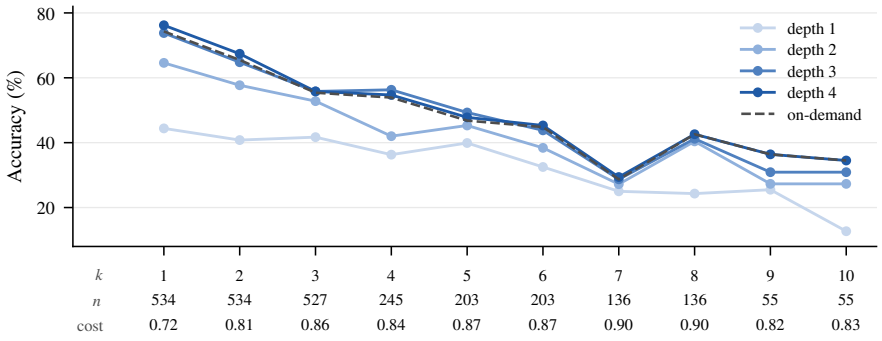


Figure 7 LEV-2.6B on LevBench by difficulty k, such as the number of reasoning steps, on the generated reasoning families and the public datasets that define one, with the number of questions n and the cost of on-demand depth below each k. Values of k above 10 cover 22 questions and are left out. Cost is the token-weighted share of the deepest pass.

7.3 RQ3: How should the depth be chosen?

Confidence thresholds set on labelled data do not beat the frontier. The usual early-exit rule stops once the top probability reaches a threshold chosen for accuracy on the labelled calibration partition. For the Ouro models, Figure 8 shows that neither this rule nor other early-exit rules, thresholds on the margin or entropy and patience-based exit (Zhou et al., 2020), beat the frontier.

Shallow confidence is overconfident and blind to fixable errors. Two properties of the shallow readouts explain this (Table 14). First, they are overconfident on new data. The questions that LEV-2.6B stops at depth one are 94.8% correct on the calibration partition but only 78.9% on Kev-Transfer, at almost the same confidence. Second, confidence separates right from wrong answers with an AUROC of 0.67 to 0.83, but separates the wrong answers that more steps fix from the rest with only 0.46 to 0.61.

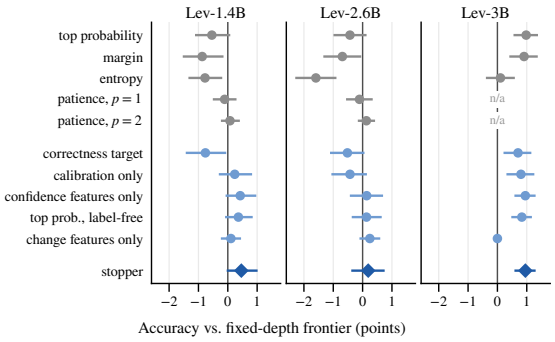


Figure 8 Accuracy of each stopping rule minus the fixed-depth frontier at the same cost on the pooled held-out panels, with 95% CIs. Gray rows are early-exit rules, and patience needs at least three steps. Light blue rows change one part of the stopper or set a single top-probability threshold the way the stopper's is set, and the change features alone never stop LEV-3B.

Setting the threshold on unlabelled requests carries the stopper. The stopper avoids the first problem by setting its threshold for agreement with the deepest answer on unlabelled requests from the panel it serves. No signal we tried overcomes the second problem, which limits how far any stopper can beat the frontier. Figure 8 shows that this carries its gain. Set the same way, a single top-probability threshold comes within 0.2 points of the stopper, so the features matter little, while a correctness target fitted on the labelled calibration partition falls below the frontier for both Ouro models. A few hundred unlabelled questions suffice, as 500 per fold bring LEV-2.6B within 0.2 points of the full fit (Section E.4). Figure 9 shows this, and a stopper fitted on one panel stays within 0.7 points of the frontier on the others unless it is fitted on the training sources.

7.4 RQ4: Does LEV beat dense cascades at the same latency?

Small LEV models fill the low-latency range. Latency is measured on the locked test partitions, and an escalated cascade request pays for both of its models (Section F). Figure 10 shows that the small LEV models open a range of latencies that no dense option covers well. At 39 to 46 ms, on-demand LEV-1.4B and LEV-3B are 7.0 to 8.8 points more accurate than the only cascade that reaches this latency, and 5.4 to 7.5 points above the best dense option. LEV-2.6B at about 150 ms is no better than Kev-4B alone.

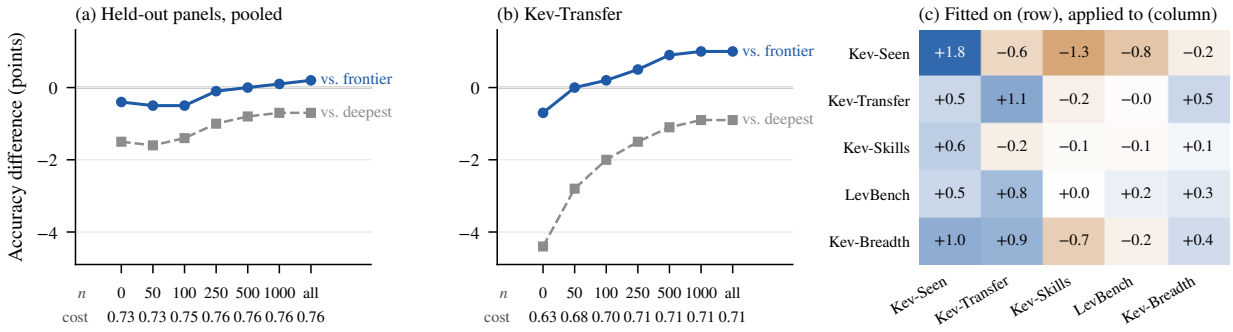


Figure 9 The stopper of LEV-2.6B with few unlabelled requests. (a, b) Fitted with n unlabelled questions per fold, as the accuracy difference in points to the deepest readout and to the fixed-depth frontier, averaged over five draws, with the cost of on-demand depth below each n. (c) Fitted on the unlabelled requests of one panel and applied to another, as the difference to the frontier. On the diagonal, it is fitted and applied on the same panel.

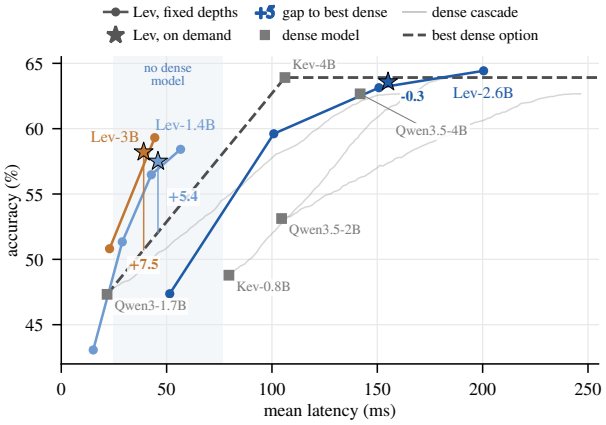


Figure 10 Accuracy against mean latency on the LevBench test partition. Lines join the fixed depths of one LEV checkpoint, and stars mark its on-demand depth. The dashed line is the best dense model or cascade, and each number is the gap of on-demand LEV to it.

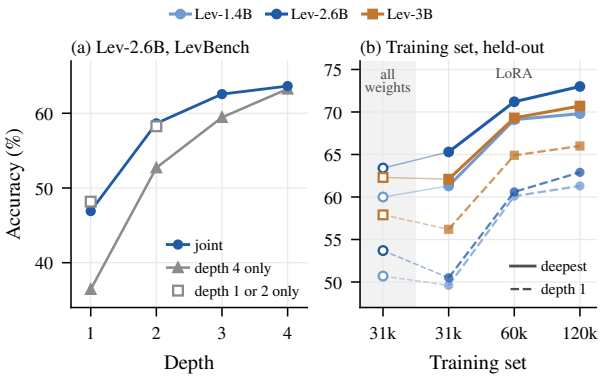


Figure 11 Training choices. (a) LEV-2.6B on LevBench, trained jointly, at depth four only, or for depth one or two alone. (b) Pooled held-out accuracy by training set. The shaded column trains all weights on Train-31k.

The advantage is in latency, not computation. Part of this advantage comes from a gap in the dense options, none of which answers between 22 and 79 ms. Counted in floating-point operations on the development partitions, every LEV model is at or below the dense options, because each loop step runs the whole backbone (Section F).

7.5 RQ5: How do training choices shape the depth curve?

Joint training serves every depth at little cost. Figure 11a compares the step-averaged loss of Equation (3) with Ouro-2.6B models trained for a single depth, and Table 21 gives the paired differences. The jointly trained LEV-2.6B is level with them at depths two and four and at most 1.3 points behind at depth one, while training only the last step leaves the shallow readouts unusable (Section G.3).

More data mainly strengthens the shallow readouts. All other results use 31,329 training records (Train-31k), which we extend to 60k and 120k (Train-60k and Train-120k, Section B.3). On the pooled held-out panels, depth one of LEV-2.6B rises from 50.5% to 62.9% and the deepest readout from 65.3% to 73.0%, so on-demand cost falls from 0.76 to 0.63 (Figure 11b). Part of this rise is in distribution, because the larger sets add sources that LevBench also draws on. On Kev-Breadth, which no training set touches,

the deepest readout stays between 67.3% and 67.7%, while Qwen3.5-2B gains 0.4 points and Qwen3.5-4B 2.0 points (Table 20).

LoRA beats full-weight fine-tuning on the Ouro models. Training all weights, shaded in Figure 11b, lowers the deepest readout of both Ouro models by 1.3 to 1.9 points and removes their gain over the frontier, while LEV-3B is unchanged (Table 19). A Brier term, step weights that grow with the step, and more loop steps than in pretraining change the deepest readout by at most 0.9 points and raise its gain over the frontier by at most 0.1 points (Table 22). The hard-tier ECE of every readout lies between 0.18 and 0.26 after temperature scaling with no trend in depth, as for the Qwen3-1.7B control, so the gap to Kev and Jev comes from our single-stage recipe rather than from the loop (Table 9).

8 Conclusion

We build LEV, a decision model that thinks in loop steps rather than tokens. One checkpoint serves several operating points whose accuracy rises with depth, mostly on hard questions. Setting the stopping threshold on unlabelled requests saves 15% to 24% of the computation at a cost of under one accuracy point, and below 50 ms the 1.4B and 3B models are 5.4 to 8.8 points more accurate than dense models and cascades of the same latency. The stopper spends more steps on harder questions without seeing their difficulty, and predicting agreement with the deepest answer works where confidence thresholds set on labelled data do not. These results suggest that a natively looped backbone turns depth into a setting chosen at serving time, so one decision model can trade latency for accuracy per question instead of per deployment. Two directions follow from the analysis. A stopping target that also recovers the questions a shallower step answers correctly could add accuracy beyond the deepest readout, and larger looped backbones would show whether the gains on hard questions grow with scale.

Limitations

Our study covers looped backbones of 1.4B to 3B parameters, the sizes that are publicly available, and applying the same recipe to larger looped models would show how far the depth curve extends. We report JevBench results on its 231 public items, and further decision benchmarks would broaden the picture. We count the cost of on-demand depth in loop steps, and a serving path that batches requests at different depths would turn this saving into wall-clock time. Finally, latency is measured one request at a time without fused kernels, graph capture or state caching for all systems, and optimised serving for looped and dense models alike is left to future work.

References

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huanru Henry Mao, Garrison W. Cottrell, and Julian McAuley. ReZero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence*, 2021.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In *International Conference on Learning Representations*, 2025.

Haowei Cao. Jev-LCT: Open system-one decision engine with free calibrated confidence from recurrent trajectories. https://github.com/gitchw/LCT, 2026.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024.

Lizhang Chen, Jonathan Li, Chen Liang, Ni Lao, and Qiang Liu. Training-free looped transformers. *arXiv preprint arXiv:2605.23872*, 2026.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *International Conference on Learning Representations*, 2020.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems*, 2025.

Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning*, 2023.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations*, 2024.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Conference on Language Modeling*, 2025.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. LoopFormer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *International Conference on Learning Representations*, 2026.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, New York, 2011.

Yeskendir Koishekenov, Aldo Lipani, and Nicola Cancedda. Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts. *arXiv preprint arXiv:2510.07358*, 2025.

Sean McLeish, Ang Li, John Kirchenbauer, Dayal Singh Kalra, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Jonas Geiping, et al. Teaching pretrained language models to think deeper with retrofitted recurrence. *arXiv preprint arXiv:2511.07384*, 2025.

Lassi Meronen, Martin Trapp, Andrea Pilzer, Le Yang, and Arno Solin. Fixing overconfidence in dynamic neural networks. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024.

Nanbeige Lab. Nanbeige4.2-3B: Unlocking agentic capabilities in a compact model. *arXiv preprint arXiv:2607.22083*, 2026.

Jared Palmer. Kev: A jev-like family of decision models. https://github.com/jaredpalmer/kev, 2026.

Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive depth in looped transformers: Diagnosing learned halting gates and trajectory readouts. *arXiv preprint arXiv:2607.20519*, 2026.

Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. Parallel context windows for large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

razorback16. OpenJev: An open, jev-compatible system one decision server on DiffusionGemma. https://github.com/razorback16/openjev, 2026.

Joshua Robinson, Christopher Michael Rytting, and David Wingate. Leveraging large language models for multiple choice question answering. In *International Conference on Learning Representations*, 2023.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, 2025.

Tal Schuster, Adam Fisch, Tommi Jaakkola, and Regina Barzilay. Consistent accelerated inference via confident adaptive transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 4962–4979, 2021.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

Mark Shapiro. Retrofitting recurrent depth into a pretrained language model: Installation, extrapolation, transfer, and retention at two parameter budgets. *arXiv preprint arXiv:2608.11233*, 2026.

Florian Standhartinger. JevBench: A benchmark and leaderboard for AI decision models. https://github.com/fstandhartinger/jevbench, 2026.

TypeSafe AI. Jev: A system one decision model. https://docs.typesafe.ai/models, 2026.

Neeraj Varshney and Chitta Baral. Model cascading: Towards jointly improving efficiency and accuracy of NLP systems. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11007–11021, 2022. doi: 10.18653/v1/2022.emnlp-main.756.

David Villalón. djev: Structured decisions with DiffusionGemma and vLLM. https://github.com/Davipar/djev-dev, 2026.

Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, 2015.

Xiaoqiang Wang, Suyuchen Wang, Yun Zhu, and Bang Liu. System-1.5 reasoning: Traversal in language and latent spaces with dynamic shortcuts. In *Advances in Neural Information Processing Systems*, 2025.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 2246–2251, 2020.

Ji Xin, Raphael Tang, Yaoliang Yu, and Jimmy Lin. BERxiT: Early exiting for BERT with better fine-tuning and extension to regression. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 91–104, 2021.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 5364–5376, 2024.

Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024.

Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. BERT loses patience: Fast and robust inference with early exit. In *Advances in Neural Information Processing Systems*, volume 33, pp. 18330–18341, 2020.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.

Appendix

Appendix Contents

A Motivation Details **14**
A.1 Routing from Kev-0.8B to Kev-4B . . . 14
A.2 Generated Reasoning . . . 15
A.3 Retrofitted Loops . . . 15

B Setup Details **16**
B.1 Training Settings . . . 16
B.2 Evaluation Panels . . . 17
B.3 Training Sets . . . 17
B.4 LevBench . . . 18
B.5 Evaluation Protocol . . . 18

C JevBench Results **19**
C.1 Tiers and Latency . . . 19
C.2 Paired Comparisons . . . 19
C.3 Calibration . . . 19

D On-Demand Depth **20**
D.1 Development Panels . . . 20
D.2 Locked Test . . . 20
D.3 Further Panels . . . 22
D.4 Tiers and Difficulty . . . 22

E Stopping Rules **23**
E.1 Stopper and Top-Probability Rule . . . 23
E.2 Shallow Confidence . . . 23
E.3 Early-Exit Rules and Stopper Variants . . . 24
E.4 Unlabelled Requests . . . 25

F Dense Models and Cascades **25**

G Training Choices **26**
G.1 Training Data and Full Weights . . . 26
G.2 Dense Controls . . . 27
G.3 Joint Training . . . 27
G.4 Losses and Loop Steps . . . 27

A Motivation Details

A.1 Routing from Kev-0.8B to Kev-4B

Figure 12 is the full routing curve behind Figure 2b. Items go from Kev-0.8B to Kev-4B in order of increasing top probability of Kev-0.8B, and the curve shows JevBench accuracy at every share of items sent. Confidence routing is above random routing once more than a seventh of the items are sent. The median top probability of Kev-0.8B is 0.80 on the items that both models answer correctly and 0.44 on the items that only Kev-4B answers correctly. The curve is flat over the last fifth, because Kev-4B changes no outcome among the fifth of items where Kev-0.8B is most confident.

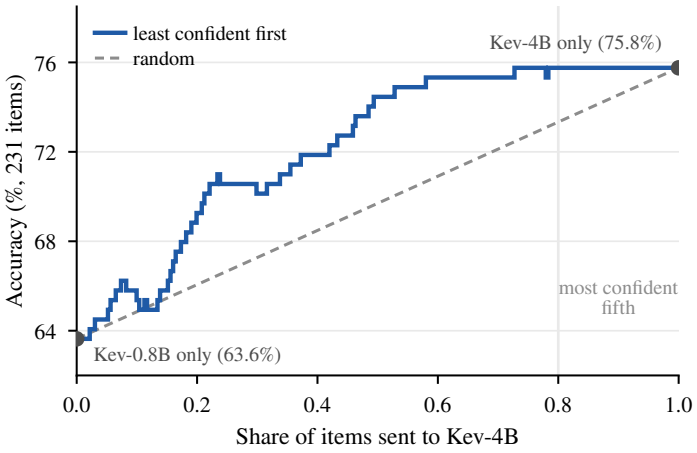


Figure 12 Accuracy on the 231 public JevBench items when the items on which Kev-0.8B is least confident are answered by Kev-4B, against random routing.

A.2 Generated Reasoning

Figure 2c takes the hard accuracy and the median latency of the hard items from the results that JevBench publishes for djev and OpenJev. Each server is measured on its own endpoint, so the latencies compare the two modes of one server but not the two servers. Both servers run DiffusionGemma 26B. djev itself always answers with a single read, so its thinking mode is a generation path over the same bf16 weights that JevBench added, while OpenJev has its own thinking option and uses NVFP4-quantised weights for the single read and the bf16 weights for thinking. The token counts are the generated tokens per decision that JevBench reports for each thinking mode. Thinking also fails in ways a single read does not. The thinking mode of djev gets 2 of the 48 easy items wrong, and with a budget of 8,192 generated tokens it returned no valid answer for 72 of the 534 decisions of the full benchmark, which also contains non-public items.

A.3 Retrofitted Loops

Figure 2d summarises Table 3, which loops layers of Qwen3.5-0.8B-Base, the backbone of Kev-0.8B. In the default loop, layers 8 to 15 of the 24-layer stack run twice. Every pass after the first is gated per channel in the ReZero style (Bachlechner et al., 2021),

$$h \leftarrow h + g_{p,i} \odot \big(f_i(h) - h\big), \tag{6}$$

where f_i is layer i and the gate $g_{p,i}$ starts at zero. At initialisation the looped model is therefore exactly the pretrained model, and training decides how much of the extra depth to use. We train a LoRA adapter and a pointer head over the options (Section 4.1) with Kev's recipe on the 12,576 records of Kev's main training partition. The recipe is two epochs of LoRA with rank 16 and a peak learning rate of 10^{-4}. The adapter is shared across passes, and each pass keeps its own attention and recurrent caches. We read the development partitions of the three tuning panels (Table 5). No variant in Table 3 improves accuracy on any panel with a confidence interval above zero. After training, the gates of the default loop have a mean magnitude of about 0.006, so the second pass contributes less than one percent of its residual update. A loop that starts closed receives little gradient to open, and a loop that starts open moves hidden states away from what the later layers were pretrained on, so removing the gate lowers accuracy.

Variant	Seeds	Kev-Seen	Kev-Transfer	Kev-Skills
Baseline accuracy	3	81.1	56.4	34.1
Default loop	3	−0.08 [−0.55, +0.40]	+0.48 [−0.29, +1.43]	+0.18 [−0.64, +0.93]
3 passes	1	−0.47 [−1.27, +0.40]	−0.29 [−1.43, +0.86]	−0.83 [−2.14, +0.55]
16 layers	1	−0.32 [−1.34, +0.63]	+0.19 [−1.24, +1.53]	−1.20 [−2.52, +0.09]
Gate lr ×3	1	+0.00 [−0.87, +0.87]	−0.57 [−1.72, +0.57]	−1.11 [−2.18, −0.09]
Gate lr ×10	1	−0.47 [−1.42, +0.40]	−0.10 [−1.34, +1.24]	−0.92 [−2.13, +0.28]
Gate lr ×30	1	+0.71 [−0.16, +1.66]	−0.10 [−1.43, +1.24]	−1.02 [−2.30, +0.19]
No gate	1	−0.55 [−1.82, +0.71]	−2.20 [−4.02, −0.48]	−0.92 [−2.80, +1.00]

Table 3 Retrofitted loops on Qwen3.5-0.8B-Base. The first row is the accuracy (%) of the unlooped baseline, and every other cell is the paired accuracy difference to it in points, with a 95% CI, on development partitions. The default loop runs layers 8–15 twice behind gates trained with a learning rate of 10^{-3}. 3 passes runs layers 8–15 three times, 16 layers loops layers 4–19, gate lr $\times N$ multiplies the gate learning rate by N, and no gate removes the gates.

B Setup Details

B.1 Training Settings

Table 4 lists the training settings. The delimiters of Section 4.1 are special tokens of the backbone's tokenizer, and user text that spells them is rewritten so that it cannot forge a boundary.

Setting	Value
Adapter and head	
LoRA rank / α / dropout	16 / 32 / 0.05
LoRA targets	attention + MLP
Pointer head dimension d_p	256
Optimisation	
Optimiser	AdamW, weight decay 0.01
Schedule	one-cycle, 10% warm-up
Peak learning rate	10^{-4}
Records per step	64
Micro-batch size	8 on average
Epochs	1
Precision	bf16 autocast
Step weights w_t (looped)	uniform, $1/R$
Temperature grid T_k	121 log-spaced in $[0.25, 4]$
Data	
Maximum state length	5,120 tokens
Records, Train-31k	31,329
Records, Train-60k	60k
Records, Train-120k	120k
Full-weight variant	
Backbone / head lr	10^{-5} / 10^{-4}
Weights / optimiser	bf16 / fp32

Table 4 Training settings of LEV and the dense controls. The step weights apply only to LEV, and the last block only to the full-weight runs.

Backbones. Ouro-1.4B is a 24-layer decoder with hidden size 2048, Ouro-2.6B has 48 layers, and Nanbeige4.2-3B runs 22 layers twice. The Qwen3-1.7B control lies between the two Ouro backbones in size, so it is the dense model of comparable size for both.

Training. Micro-batches group records of similar length so that short records share a pass and long records run alone. Each optimiser step still sees 64 records. With probability 0.25, Kev's "none of the above" augmentation adds such an option to a choice question and trains two copies of the record, one with the correct option and one without it, where "none of the above" is correct.

B.2 Evaluation Panels

Table 5 maps each panel to its source suite. The three tuning panels and Kev-Breadth are development partitions of frozen Kev suites, and LevBench is the suite we built for this study. The calibration partition is not a panel. It fits the temperature of each depth and the thresholds of the early-exit rules, and it is part of the data the stopper is fitted on. The locked test uses the test partitions of LevBench (8,740 scored questions) and Kev-Breadth (3,031), and also reports the test partitions of the suites behind the three tuning panels.

Group	Panel	Suite, partition	Questions	Content
Tuning	Kev-Seen	`decision-v7`, dev.	1,464	Ten public classification and inference datasets, generated policies and rules
	Kev-Transfer	`transfer-v9`, dev.	1,264	New sources such as MMLU-Pro and PAWS, held-out rule families, unknowable items
	Kev-Skills	`hard-v1`, dev.	1,083	Seven programmatically labelled skill families, held-out templates
Held-out	LevBench	ours, dev.	4,370	Tiers *id*, *near*, *deep*, and *far* by distance from the training data
	Kev-Breadth	`breadth-v1`, dev.	3,023	14 datasets from five application areas, never trained on
Benchmark	JevBench	public items	231	48 easy, 72 standard, and 111 hard items
Calibration	–	decision-v7, calib.	1,148	Temperatures per depth, and part of the data the stopper is fitted on

Table 5 Evaluation panels and the suites they come from. Suite names are those of the Kev repository. Questions are the scored questions of each panel.

B.3 Training Sets

Base training set. Train-31k combines Kev's main training partition (12,576 records from public classification and inference datasets, generated policies, and generated rule structures), generated records that state day counts or omit the deciding evidence (2,225), Kev's generated skill families (6,000), developer-tooling decisions (5,320), and consumer-complaint documents (5,219). After removing records whose state exceeds 5,120 tokens, 31,329 records remain.

Larger training sets. Train-60k and Train-120k contain Train-31k and add records from a pool that we screened against every evaluation partition we use, including the public JevBench items. The pool holds 32,000 records from four generated reasoning families (rule application, dependency chains, ledger steps, and date arithmetic), each with a difficulty k inside its training range and 10% of them unknowable, 12,000 records from new seeds of the skill generators, 16,000 further records from the public sources of Kev-Seen, 7,721 records from ARC, OpenBookQA, and CommonsenseQA, and 22,002 records from CondaQA, Social IQa, CosmosQA, MathQA, QASC, and PIQA. Duplicates and records that match an evaluation item are removed. We interleave the pool by part so that every prefix keeps the proportions of the parts. Train-60k and Train-120k add its first 28,660 and 88,660 records, so each larger set contains the smaller one with the same mix (Table 6).

Part of the pool	Train-60k	Train-120k
Generated reasoning families	10,222	31,621
Six further public datasets	7,028	21,741
Public sources of Kev-Seen	5,111	15,810
New seeds, skill generators	3,833	11,858
ARC, OpenBookQA, CommonsenseQA	2,466	7,630
All	28,660	88,660

Table 6 Records that the larger training sets add to Train-31k, by part of the pool.

B.4 LevBench

The development partition has 4,870 questions in five tiers, of which 4,370 are scored for accuracy. The *id* tier (1,370 questions) holds new items of the sources of Train-120k: 970 from its public datasets, 520 of them from sources that Train-31k also trains on, and 400 from the four generated families with a difficulty inside the training range. The *near* and *deep* tiers (800 questions each) hold a held-out template of the four generated families above, with a difficulty inside and beyond the range that Train-60k and Train-120k train on. Train-31k trains on none of these families, so for Train-31k they are new tasks. The *far* tier holds eight datasets that never enter training: BoardgameQA, StepGame, CLadder, LogiQA 2.0, AR-LSAT, QuAIL, TimeDial, and StrategyQA. We score 1,400 of its questions and leave out its 200 StrategyQA questions. The unknowable tier (300 questions) removes the deciding fact from generated questions and is scored only by how often the model answers confidently. Every item was screened against Train-31k, all evaluation partitions of the Kev suites, the public JevBench items, and MMLU-Pro. A public item was removed on any match, and a generated item on an exact match. The suite was frozen before any model read it.

B.5 Evaluation Protocol

Evaluator. Our evaluator builds each item's request with JevBench's own question builder, passes it through the same request conversion that a Kev server uses, and scores the answer with JevBench's scoring code. Table 7 compares it with the official JevBench harness on the released Kev checkpoints. On Kev-4B it reproduces the harness exactly, and on Kev-0.8B it matches overall accuracy and differs by under one point on the hard tier. For a looped model it reads every depth from one forward pass.

	Kev-0.8B		Kev-4B	
	Ours	Harness	Ours	Harness
Acc. (%)	63.6	63.6	75.8	75.8
Hard acc. (%)	36.9	36.0	54.1	54.1
Hard ECE	0.172	0.181	0.112	0.112

Table 7 Our in-process evaluator against the official JevBench harness on the released Kev checkpoints.

Metrics. The Intelligence score of Table 8 follows JevBench version 1.3. It is a weighted mean over tiers of the chance-corrected accuracy $(a - c)/(1 - c)$, where a is the accuracy on a tier and c the accuracy of random guessing on it, with the official tier weights renormalised over the three public tiers. The calibration error is the top-label ECE. Accuracy is computed in fp32. On JevBench we add an exact McNemar test to the paired bootstrap.

Latency. Latency in Tables 1 and 8 is the median over the 231 JevBench items of one request at a time, after a short warm-up.

Jev. Jev's per-item outcomes are taken from the outcomes published in the JevBench repository, and on Kev-Skills and Kev-Breadth from the reads published in the Kev repository.

C JevBench Results

C.1 Tiers and Latency

Table 8 splits JevBench by tier (Section 7.1).

System	Params	Depth	Latency (ms)	Easy	Standard	Hard	All	Intelligence
LEV, one checkpoint per model read at every depth								
LEV-1.4B Ouro-1.4B	1.4B	1	**15**	100.0	70.8	31.5	58.0	41.7
		2	30	100.0	80.6	40.5	65.4	51.6
		3	44	100.0	81.9	45.0	68.0	55.2
		4	58	97.9	84.7	44.1	68.0	55.7
LEV-2.6B Ouro-2.6B	2.6B	1	51	97.9	76.4	39.6	63.2	48.1
		2	101	100.0	88.9	46.8	71.0	60.3
		3	150	100.0	88.9	55.9	75.3	66.0
		4	200	100.0	**90.3**	**56.8**	**76.2**	**67.3**
LEV-3B Nanbeige4.2-3B	3B	1	23	100.0	83.3	46.8	69.3	57.1
		2	45	100.0	**90.3**	49.5	72.7	62.8
Dense controls, same data and recipe								
Qwen3-1.7B	1.7B	–	22	100.0	84.7	34.2	63.6	50.0
Qwen3.5-2B	2B	–	105	100.0	88.9	42.3	68.8	57.5
Qwen3.5-4B	4B	–	140	100.0	**90.3**	51.4	73.6	63.9
Released references, trained differently								
Kev-0.8B	0.8B	–	79	100.0	80.6	36.9	63.6	49.3
Kev-4B	4B	–	108	100.0	93.1	54.1	75.8	67.2
Jev 1.13.0	n/a	–	652†	100.0	98.6	73.0	86.6	82.2

Table 8 Accuracy and latency on the public JevBench items per tier (48 easy, 72 standard, 111 hard), with the JevBench Intelligence score. Bold marks the best value among the models trained on identical data.
†Measured through the production API with the network included, and not comparable to the local measurements.

Figure 4 places the LEV checkpoints and the dense models on the 231 public JevBench items (Section 6).

C.2 Paired Comparisons

Figure 13 shows the paired differences on JevBench behind Section 6. The McNemar test gives $p < 0.001$ for depth four against depth one of LEV-2.6B and $p = 0.002$ for LEV-1.4B. LEV-2.6B is ahead of LEV-1.4B by 8.2 points at depth four (95% CI $[+3.5, +13.0]$) and by 12.6 points on the hard tier ($[+4.5, +21.6]$).

C.3 Calibration

Table 9 lists the hard-tier ECE of every system, with the LEV readouts and the control at their fitted temperatures (Section 7.5). The fitted temperatures lie between 1.05 and 1.18 for the Ouro readouts and are 1.00 and 1.02 for LEV-3B, so on the calibration partition the raw readouts are at most slightly overconfident. Temperature scaling lowers the hard-tier ECE at depth four from 0.255 to 0.231 for LEV-1.4B and from 0.218 to 0.192 for LEV-2.6B, and that of the Qwen3-1.7B control from 0.258 to 0.253. After scaling, the ECE of LEV-1.4B falls from depth one to depth three and rises slightly at depth four, and LEV-2.6B has its lowest ECE at depth one and highest at depth two.

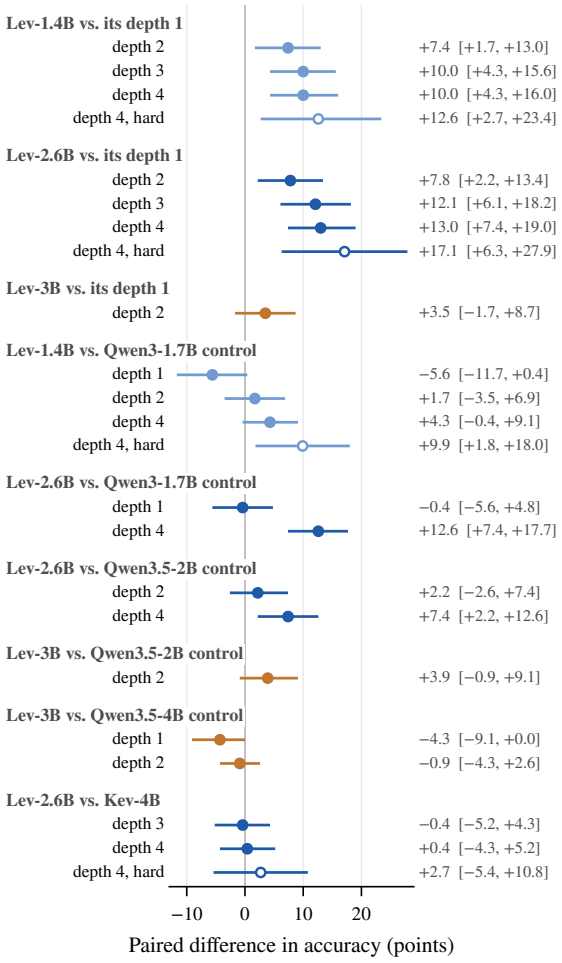


Figure 13 Paired item-level differences in accuracy on the public JevBench items (231 in all, 111 hard), in points with 95% CIs. Open markers are the hard items.

System	Depth	Hard ECE	Hard acc.
Lev, one checkpoint per model read at every depth			
Lev-1.4B	1	0.255	31.5
	2	0.237	40.5
	3	0.226	45.0
	4	0.231	44.1
Lev-2.6B	1	0.183	39.6
	2	0.241	46.8
	3	0.202	55.9
	4	0.192	56.8
Lev-3B	1	0.219	46.8
	2	0.221	49.5
Dense control, same data and recipe			
Qwen3-1.7B	–	0.253	34.2
Released references, trained differently			
Kev-0.8B	–	0.172	36.9
Kev-4B	–	0.112	54.1
Jev 1.13.0	–	0.086	73.0

Table 9 Top-label expected calibration error on the 111 hard JevBench items, with the Lev readouts and the control at their fitted temperatures.

D On-Demand Depth

Table 2 in Section 7.2 gives the pooled held-out numbers on development and on the locked test, and this section gives every panel.

D.1 Development Panels

Table 10 gives on-demand depth on every development panel. An oracle shows the headroom. On the tuning panels, stopping each question at the earliest depth that answers it correctly would reach 85.9% at a cost of 0.34, against 77.4% for the deepest readout. The temperature per depth matters only for calibration. With one temperature for all depths or none, on-demand accuracy on the tuning and held-out panels changes by at most 0.2 points.

D.2 Locked Test

Table 11 gives on-demand depth on every test partition, read once with the final configuration, and Table 2 repeats its pooled held-out rows. The test partitions of the three tuning suites are reported but did not enter any decision.

Model	Panel	Deepest	On-demand	Cost	vs. deepest		vs. frontier	
LEV-1.4B	LevBench	58.1	57.5	0.80	−0.6	[−1.0, −0.3]	+0.8	[+0.1, +1.4]
	Kev-Breadth	66.0	66.1	0.75	+0.1	[−0.4, +0.6]	−0.2	[−1.1, +0.7]
	held-out, pooled	61.3	61.0	0.78	−0.3	[−0.6, −0.0]	+0.5	[−0.0, +1.0]
	JevBench	68.0	67.1	0.59	−0.9	[−2.6, +0.9]	+0.8	[−2.2, +4.0]
LEV-2.6B	Kev-Seen	86.1	85.9	0.44	−0.2	[−1.0, +0.5]	+1.8	[+0.6, +3.1]
	Kev-Transfer	72.5	71.5	0.71	−0.9	[−1.8, −0.2]	+1.0	[−0.4, +2.5]
	Kev-Skills	71.4	70.5	0.85	−0.9	[−1.7, −0.2]	−0.3	[−1.5, +0.7]
	tuning, pooled	77.4	76.7	0.64	−0.7	[−1.1, −0.2]	+1.4	[+0.6, +2.1]
	LevBench	63.6	62.8	0.76	−0.9	[−1.3, −0.5]	+0.1	[−0.6, +0.9]
	Kev-Breadth	67.7	67.3	0.76	−0.4	[−0.9, +0.2]	+0.3	[−0.6, +1.0]
	held-out, pooled	65.3	64.6	0.76	−0.7	[−1.0, −0.4]	+0.2	[−0.4, +0.8]
	JevBench	76.2	74.0	0.61	−2.2	[−4.3, −0.4]	+1.2	[−2.4, +4.4]
LEV-3B	LevBench	59.4	58.5	0.87	−0.9	[−1.4, −0.6]	+1.0	[+0.5, +1.4]
	Kev-Breadth	66.1	65.4	0.82	−0.7	[−1.2, −0.2]	+0.6	[+0.1, +1.2]
	held-out, pooled	62.1	61.3	0.85	−0.8	[−1.1, −0.5]	+1.0	[+0.6, +1.3]
	JevBench	72.7	71.4	0.70	−1.3	[−3.5, +0.4]	+0.8	[−2.2, +3.7]

Table 10 On-demand depth with the stopper of Section 4.3. Cost is the token-weighted share of the deepest pass. The last four columns are paired differences in accuracy points with 95% CIs, against the deepest readout and against the fixed-depth frontier at the same cost. Only LEV-2.6B is read on the tuning panels.

Model	Panel (test)	Deepest	On-demand	Cost	vs. deepest		vs. frontier	
LEV-1.4B	LevBench	58.4	57.5	0.81	−0.9	[−1.2, −0.6]	+0.5	[+0.1, +1.0]
	Kev-Breadth	64.9	64.7	0.75	−0.3	[−0.8, +0.2]	+0.3	[−0.6, +1.2]
	held-out, pooled	60.1	59.3	0.80	−0.7	[−1.0, −0.5]	+0.5	[+0.1, +0.9]
	Kev-Seen	85.3	84.0	0.50	−1.3	[−2.2, −0.5]	−0.3	[−1.7, +1.1]
	Kev-Transfer	70.6	68.4	0.75	−2.2	[−3.2, −1.4]	+0.2	[−1.6, +2.0]
	Kev-Skills	70.5	69.5	0.86	−1.0	[−1.8, −0.3]	−0.5	[−1.5, +0.5]
LEV-2.6B	LevBench	64.4	63.6	0.78	−0.8	[−1.1, −0.6]	+0.3	[−0.2, +0.8]
	Kev-Breadth	65.3	64.2	0.75	−1.0	[−1.5, −0.6]	−0.5	[−1.4, +0.3]
	held-out, pooled	64.6	63.7	0.78	−0.9	[−1.1, −0.6]	+0.1	[−0.4, +0.5]
	Kev-Seen	87.3	86.2	0.44	−1.0	[−1.9, −0.2]	+1.4	[+0.4, +2.5]
	Kev-Transfer	75.3	74.3	0.70	−1.0	[−2.0, −0.2]	+0.6	[−0.7, +2.0]
	Kev-Skills	73.4	72.2	0.80	−1.2	[−2.0, −0.4]	−0.5	[−1.9, +0.9]
LEV-3B	LevBench	59.3	58.2	0.88	−1.1	[−1.4, −0.8]	+0.9	[+0.6, +1.2]
	Kev-Breadth	64.8	64.2	0.82	−0.5	[−1.1, −0.0]	+0.5	[−0.1, +1.1]
	held-out, pooled	60.7	59.8	0.87	−1.0	[−1.2, −0.7]	+0.9	[+0.6, +1.2]
	Kev-Seen	85.8	84.7	0.61	−1.1	[−2.1, −0.3]	+0.4	[−0.6, +1.2]
	Kev-Transfer	72.2	70.8	0.78	−1.3	[−2.3, −0.5]	+2.5	[+1.4, +3.7]
	Kev-Skills	69.5	69.2	0.89	−0.3	[−1.1, +0.6]	+0.7	[−0.2, +1.5]

Table 11 On-demand depth on the test partitions, in the format of Table 10. The pooled row combines the two held-out panels.

D.3 Further Panels

Table 12 adds the development partitions of two training sources, consumer-complaint documents (`documents-v1`, 920 questions) and developer-tooling decisions (`devtools-v1`, 1,074), and 1,000 MMLU-Pro questions from a public external set that no training set includes. As on the tiers of LevBench (Section 7.2), every model computes less on the two training sources than on MMLU-Pro.

Model	Panel	Deepest	On-demand	Cost	vs. deepest		vs. frontier	
LEV-1.4B	`documents-v1`	87.1	87.0	0.57	−0.1	[−1.0, +0.8]	+0.0	[−1.2, +1.3]
	`devtools-v1`	67.0	66.4	0.67	−0.7	[−1.5, +0.1]	+0.1	[−1.3, +1.4]
	MMLU-Pro	38.6	37.6	0.88	−1.0	[−1.8, −0.3]	−0.6	[−1.7, +0.5]
LEV-2.6B	`documents-v1`	88.7	87.9	0.43	−0.8	[−1.7, +0.2]	+2.1	[+1.0, +3.2]
	`devtools-v1`	69.2	68.1	0.74	−1.1	[−1.9, −0.4]	−0.1	[−1.4, +1.2]
	MMLU-Pro	48.7	47.9	0.83	−0.8	[−1.6, −0.1]	+0.3	[−1.0, +1.6]
LEV-3B	`documents-v1`	88.3	87.5	0.65	−0.8	[−1.6, +0.0]	+2.3	[+1.2, +3.5]
	`devtools-v1`	69.1	67.4	0.82	−1.7	[−2.8, −0.6]	−0.3	[−1.2, +0.7]
	MMLU-Pro	46.6	46.2	0.92	−0.4	[−1.1, +0.3]	+1.1	[+0.4, +1.9]

Table 12 On-demand depth on further panels, in the format of Table 10.

D.4 Tiers and Difficulty

Table 13 splits on-demand depth of LEV-2.6B by the tiers of LevBench (Section 7.2).

Tier	Depth 1 / 4	On-demand	Cost	vs. frontier
id	62.6 / 75.8	75.3	0.66	+1.8 [+0.6, +3.0]
near	42.0 / 71.8	69.8	0.79	−0.3 [−2.4, +1.4]
deep	34.5 / 45.4	44.9	0.87	−0.3 [−1.5, +0.9]
far	41.5 / 57.5	56.8	0.75	−0.2 [−1.6, +1.2]

Table 13 On-demand depth of LEV-2.6B on the tiers of LevBench, ordered by distance from the training data.

Figure 7 in Section 7.2 breaks the LevBench questions that carry a difficulty k down by k, and Figure 14 reports how often LEV-2.6B is confident on its 300 unknowable questions.

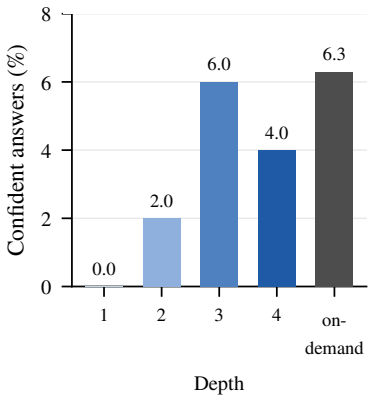


Figure 14 Share of the 300 unknowable LevBench questions that LEV-2.6B answers with a top probability of at least 0.9, by depth and on demand.

E Stopping Rules

E.1 Stopper and Top-Probability Rule

Stopper. Each per-depth stopper is a logistic regression with scikit-learn's default L2 regularisation on the eight features of Section 4.3, computed from readouts at their fitted temperatures. At depth one there is no earlier readout, so the features that compare with depth $k-1$ take the values of an unchanged readout. The threshold is searched over 201 values in $[0.5, 1]$. The two folds of each panel are assigned by a hash of the record group, so related questions share a fold. Fitted on the calibration partition alone, the stopper has the threshold 0.915 for both Ouro models and 0.8075 for LEV-3B.

Top-probability rule. A question stops at the first depth whose top probability, after temperature scaling, reaches a threshold. We choose this threshold on the calibration partition as the cheapest threshold whose accuracy is within one point of the deepest readout, which gives 0.81 for LEV-2.6B, 0.76 for LEV-1.4B, and 0.69 for LEV-3B.

E.2 Shallow Confidence

Table 14 gives the diagnosis of Section 7.3 for LEV-2.6B on every panel. The stopped questions are those whose depth-one top probability reaches the threshold of the top-probability rule. The first AUROC separates right from wrong depth-one answers by the depth-one top probability. The second does so among the questions that depth one gets wrong, for those that the deepest readout fixes against those it does not.

	Stopped at depth 1		AUROC	
Panel	Top prob.	Acc.	Right/wrong	Fixed/not
Calibration	95.3	94.8	0.82	0.50
Kev-Seen	95.5	94.7	0.83	0.46
Kev-Transfer	93.3	78.9	0.72	0.60
Kev-Skills	88.8	81.6	0.67	0.57
LevBench	93.0	88.2	0.73	0.61
Kev-Breadth	90.0	77.1	0.70	0.49

Table 14 Depth-one readout of LEV-2.6B per panel. The first two columns, in percent, describe the questions that the top-probability rule stops at depth one.

Table 15 gives the second AUROC for every checkpoint. More training data does not raise it. On Kev-Breadth, it stays between 0.39 and 0.53 for every backbone, training set, and full-weight variant. The ECE on Kev-Breadth lies between 0.07 and 0.12 for every readout, and neither more training data nor full-weight fine-tuning lowers it consistently.

	AUROC		ECE, Kev-Breadth	
Training set	LevBench	Kev-Breadth	Depth 1	Deepest
LEV-1.4B				
Train-31k	0.59	0.53	0.11	0.08
Train-60k	0.58	0.49	0.11	0.10
Train-120k	0.59	0.50	0.12	0.12
full	0.56	0.49	0.12	0.09
LEV-2.6B				
Train-31k	0.61	0.49	0.10	0.09
Train-60k	0.64	0.47	0.08	0.07
Train-120k	0.58	0.49	0.10	0.11
full	0.61	0.45	0.07	0.07
LEV-3B				
Train-31k	0.54	0.43	0.12	0.10
Train-60k	0.54	0.43	0.09	0.09
Train-120k	0.51	0.39	0.10	0.10
full	0.49	0.42	0.09	0.08

Table 15 How well the depth-one top probability finds wrong answers that more steps fix, and the calibration of the shallowest and deepest readouts on Kev-Breadth. Gray rows (full) train all weights on Train-31k.

E.3 Early-Exit Rules and Stopper Variants

Table 16 gives the accuracy and cost behind Figure 8 in Section 7.3, which compares the stopping rules with the fixed-depth frontier on the pooled held-out panels.

	LEV-1.4B			LEV-2.6B			LEV-3B		
Rule	Acc.	Cost	vs. frontier	Acc.	Cost	vs. frontier	Acc.	Cost	vs. frontier
Rules from the early-exit literature									
top probability	60.1	0.79	−0.5 [−1.1, +0.1]	64.2	0.81	−0.4 [−1.0, +0.1]	60.9	0.81	+1.0 [+0.5, +1.4]
margin	59.6	0.75	−0.9 [−1.5, −0.1]	63.8	0.78	−0.7 [−1.3, −0.0]	60.3	0.77	+0.9 [+0.4, +1.4]
entropy	59.8	0.80	−0.8 [−1.3, −0.2]	62.8	0.76	−1.6 [−2.3, −0.9]	58.2	0.66	+0.1 [−0.4, +0.6]
patience, $p = 1$	58.7	0.62	−0.1 [−0.5, +0.3]	62.9	0.63	−0.1 [−0.6, +0.3]	–	–	–
patience, $p = 2$	60.9	0.87	+0.1 [−0.2, +0.4]	65.0	0.88	+0.1 [−0.2, +0.4]	–	–	–
Stopper variants									
correctness target	59.7	0.75	−0.8 [−1.4, −0.1]	64.0	0.80	−0.5 [−1.1, +0.1]	60.1	0.77	+0.7 [+0.2, +1.2]
calibration only	60.7	0.76	+0.2 [−0.3, +0.8]	63.8	0.73	−0.4 [−1.1, +0.1]	60.2	0.77	+0.8 [+0.3, +1.3]
confidence features only	61.0	0.78	+0.4 [−0.1, +1.0]	64.6	0.76	+0.1 [−0.4, +0.7]	61.3	0.85	+1.0 [+0.6, +1.3]
top probability, label-free	61.0	0.80	+0.4 [−0.1, +0.8]	64.6	0.78	+0.1 [−0.4, +0.7]	61.3	0.86	+0.8 [+0.5, +1.2]
change features only	60.9	0.86	+0.1 [−0.2, +0.5]	65.0	0.85	+0.2 [−0.1, +0.6]	62.1	1.00	+0.0 [+0.0, +0.0]
stopper	61.0	0.78	+0.5 [−0.0, +1.0]	64.6	0.76	+0.2 [−0.4, +0.8]	61.3	0.85	+1.0 [+0.6, +1.3]

Table 16 Stopping rules and stopper variants on the pooled held-out panels, with the accuracy difference in points to the fixed-depth frontier at the same cost, and its 95% CI. The label-free top-probability row sets one threshold the way the stopper's is set.

Rules and variants. The margin and entropy rules choose their thresholds as the top-probability rule does. Patience-based exit stops once the answer has not changed for p steps, so it needs at least three steps. The confidence features are the top probability, the margin, the normalised entropy, the log of the number of options, and the depth. The change features are the agreement with the previous answer, the KL divergence from the previous readout, the change of the top probability, and the depth. With two steps there is no earlier

answer at depth one, so the change features alone never stop LEV-3B. The correctness target predicts whether the answer at the current depth is right and is fitted on the labelled calibration partition, with its threshold chosen as for the top-probability rule. The label-free top-probability row keeps only the top probability as the stopper's feature, so its threshold on that score is set as the stopper's is, on unlabelled requests for 98% agreement with the deepest answer.

Results. On the tuning panels, the top-probability rule of LEV-2.6B reaches 75.8% at a cost of 0.69, 0.2 points below the frontier ($[-1.0, +0.6]$). The calibration-only stopper reaches 75.6% at 0.61 ($+0.6$, $[-0.3, +1.4]$), and the stopper 76.7% at 0.64 ($+1.4$, $[+0.6, +2.1]$). Against random escalation with the same distribution of stopping depths, the top-probability rule of LEV-2.6B gains 3.8 points on Kev-Seen ($[+2.6, +5.0]$), 2.5 on Kev-Transfer ($[+1.0, +4.0]$), and 2.2 on Kev-Skills ($[+1.1, +3.3]$). For LEV-3B, whose two steps make the frontier a straight line, every rule that stops some questions at depth one, except the entropy threshold, is above it.

E.4 Unlabelled Requests

Figure 9a and b fit the stopper of LEV-2.6B with only n unlabelled questions from the other fold, drawn five times, and $n = 0$ fits it on the calibration partition alone. With $n = 0$ the stopper of LEV-2.6B stops too early on the held-out panels and falls 1.5 points below the deepest readout, against 0.7 points with all questions of the other fold. Figure 9c instead fits it on all unlabelled requests of one panel and applies it to another. A stopper fitted on the training sources (Kev-Seen) falls 0.2 to 1.3 points below the frontier on every other panel, while one fitted on any other panel is at most 0.7 points below it anywhere.

F Dense Models and Cascades

Table 17 lists the comparisons behind Figure 10 in Section 7.4, with their confidence intervals, and the same comparisons on the development partitions.

		Test		Development	
Model	Compared with	LevBench	Kev-Breadth	LevBench	Kev-Breadth
LEV-1.4B	Qwen3-1.7B → Qwen3.5-4B	+7.0 [+5.9, +8.1]	+5.5 [+3.9, +7.0]	+7.0 [+5.5, +8.4]	+6.6 [+4.9, +8.2]
	best dense option	+5.4 [+4.5, +6.3]	+4.5 [+3.1, +5.8]	+5.2 [+3.9, +6.4]	+5.5 [+4.1, +6.9]
LEV-2.6B	Qwen3.5-2B → Qwen3.5-4B	+6.2 [+5.2, +7.3]	−1.2 [−2.7, +0.3]	+6.1 [+4.6, +7.7]	+0.6 [−0.8, +2.1]
	Qwen3-1.7B → Qwen3.5-4B	+1.0 [+0.0, +1.9]	−2.4 [−3.9, −1.0]	+0.8 [−0.6, +2.2]	−0.0 [−1.4, +1.2]
	Kev-0.8B → Kev-4B	+2.1 [+1.0, +3.2]	−1.4 [−3.0, +0.2]	+2.7 [+1.3, +4.1]	+0.8 [−0.7, +2.2]
	LEV-1.4B (depth 4) → LEV-2.6B	+1.2 [+0.7, +1.8]	−1.6 [−2.3, −0.8]	+1.3 [+0.5, +2.1]	−0.3 [−1.1, +0.5]
	best dense option	−0.3 [−1.3, +0.7]	−2.6 [−4.1, −1.1]	−0.6 [−2.0, +0.8]	−0.5 [−1.9, +0.8]
LEV-3B	Qwen3-1.7B → Qwen3.5-4B	+8.8 [+7.7, +9.9]	+5.3 [+3.6, +6.9]	+8.6 [+7.1, +10.1]	+6.2 [+4.4, +8.0]
	best dense option	+7.5 [+6.5, +8.4]	+4.6 [+3.2, +6.1]	+7.3 [+6.0, +8.6]	+5.4 [+3.9, +7.0]
Best dense option, at equal floating-point operations					
LEV-1.4B		–	–	−6.0 [−7.4, −4.6]	−1.8 [−3.2, −0.3]
LEV-2.6B		–	–	−0.7 [−2.1, +0.7]	−0.6 [−1.9, +0.8]
LEV-3B		–	–	−5.0 [−6.4, −3.6]	−2.5 [−3.9, −1.1]

Table 17 Accuracy of on-demand LEV minus a cascade or the best dense option at the same mean latency, in points with 95% CIs. On the LevBench test partition, on-demand LEV-1.4B, LEV-2.6B, and LEV-3B take 46, 155, and 39 ms on average. A cascade that cannot reach a model's latency is omitted. The last rows compare at equal floating-point operations instead of latency.

Latencies come from one H200 GPU in bf16, one request at a time, without fused kernels or graph capture. For each system and depth we time 600 development records three times, take the median, and fit latency as a linear function of the token count, which then prices every request. An on-demand request pays the latency

of the deepest step its state runs. A cascade escalates with the top-probability rule of Section 7.3 applied to the small model, and an escalated request pays for both models. A stopper of the same kind as LEV's, which predicts whether the small and the large model agree, leads to the same conclusions. We compare at the mean latency of on-demand LEV and mix the two neighbouring operating points of a cascade to reach it. The best dense option is the upper envelope of every dense model and every dense cascade. Counted in floating-point operations, a request costs two operations per parameter, token, and loop step.

G Training Choices

G.1 Training Data and Full Weights

Table 19 gives the results on the pooled held-out panels, and Table 18 splits them by panel. The full-weight variant uses a ten times lower learning rate for the backbone than LoRA and keeps the same data, schedule, and number of steps.

		LevBench				Kev-Breadth				JevBench
Model	Training set	Deepest	On-demand	Cost	vs. frontier	Deepest	On-demand	Cost	vs. frontier	Deepest
LEV-1.4B	Train-31k	58.1	57.5	0.80	+0.8 [+0.1, +1.4]	66.0	66.1	0.75	−0.2 [−1.1, +0.7]	68.0
	Train-60k	70.8	70.0	0.67	+0.5 [−0.2, +1.2]	66.7	65.9	0.75	+0.1 [−0.7, +1.0]	69.7
	Train-120k	73.0	71.9	0.61	−0.4 [−1.1, +0.3]	65.2	64.8	0.75	−0.3 [−1.2, +0.5]	68.0
	full	57.9	56.8	0.73	+0.1 [−0.7, +0.8]	63.0	63.0	0.74	−0.4 [−1.1, +0.4]	68.8
LEV-2.6B	Train-31k	63.6	62.8	0.76	+0.1 [−0.6, +0.9]	67.7	67.3	0.76	+0.3 [−0.6, +1.0]	76.2
	Train-60k	73.8	72.7	0.66	+0.4 [−0.3, +1.2]	67.3	67.1	0.73	+0.6 [−0.2, +1.4]	75.3
	Train-120k	77.0	76.1	0.59	+1.5 [+0.7, +2.3]	67.3	66.6	0.70	−0.3 [−1.0, +0.4]	77.9
	full	61.6	60.8	0.76	−0.7 [−1.4, −0.0]	66.0	65.6	0.67	−0.1 [−0.8, +0.6]	76.6
LEV-3B	Train-31k	59.4	58.5	0.87	+1.0 [+0.5, +1.4]	66.1	65.4	0.82	+0.6 [+0.1, +1.2]	72.7
	Train-60k	71.2	70.5	0.79	+1.9 [+1.4, +2.5]	66.5	66.5	0.81	+0.7 [+0.1, +1.2]	74.0
	Train-120k	73.6	72.6	0.74	+2.2 [+1.6, +2.8]	66.5	66.4	0.81	+0.9 [+0.3, +1.5]	74.9
	full	59.7	58.9	0.83	+1.1 [+0.6, +1.5]	66.1	65.9	0.79	+0.9 [+0.4, +1.5]	74.9

Table 18 Table 19 per panel, with the deepest JevBench accuracy of each checkpoint. Gray rows (full) train all weights on Train-31k.

Training set	Depth 1	Deepest	Cost	vs. frontier
LEV-1.4B				
Train-31k	49.6	61.3	0.78	+0.5 [−0.0, +1.0]
Train-60k	60.1	69.1	0.70	+0.3 [−0.2, +0.8]
Train-120k	61.3	69.8	0.67	−0.4 [−0.9, +0.1]
full	50.7	60.0	0.73	−0.1 [−0.7, +0.4]
LEV-2.6B				
Train-31k	50.5	65.3	0.76	+0.2 [−0.4, +0.8]
Train-60k	60.6	71.2	0.69	+0.4 [−0.1, +1.0]
Train-120k	62.9	73.0	0.63	+0.7 [+0.1, +1.2]
full	53.7	63.4	0.72	−0.3 [−0.8, +0.1]
LEV-3B				
Train-31k	56.2	62.1	0.85	+1.0 [+0.6, +1.3]
Train-60k	64.9	69.3	0.80	+1.4 [+0.9, +1.8]
Train-120k	66.0	70.7	0.77	+1.5 [+1.1, +2.0]
full	57.9	62.3	0.81	+1.1 [+0.7, +1.4]

Table 19 Training data and full-weight fine-tuning, on the pooled held-out panels. Cost and the difference to the frontier are for on-demand depth. Gray rows (full) train all weights on Train-31k. Part of LevBench is in distribution for Train-60k and Train-120k (Section B.4).

G.2 Dense Controls

We train the dense controls on the three training sets with the same recipe and compare them in Table 20 with the deepest readout of the looped model of about the same size.

	LevBench			Kev-Breadth		
Model	31k	60k	120k	31k	60k	120k
LEV-2.6B	63.6	73.8	77.0	67.7	67.3	67.3
Qwen3.5-2B	53.1	66.3	69.9	64.7	64.3	65.1
LEV-3B	59.4	71.2	73.6	66.1	66.5	66.5
Qwen3.5-4B	62.1	71.8	74.2	67.4	69.4	69.4

Table 20 Accuracy of the deepest readout of the looped models and of dense controls trained on Train-31k, Train-60k, and Train-120k (columns). Part of LevBench is in distribution for Train-60k and Train-120k.

G.3 Joint Training

Table 21 gives the numbers behind Figure 11a in Section 7.5, with the paired difference between joint and single-depth training.

	LevBench			Kev-Breadth		
Depth	Joint	Single	Δ	Joint	Single	Δ
1	46.9	48.2	−1.3 [−2.5, +0.0]	55.7	56.9	−1.2 [−2.6, +0.3]
2	58.6	58.3	+0.4 [−0.6, +1.4]	65.3	64.7	+0.6 [−0.4, +1.7]
4	63.6	63.2	+0.4 [−0.6, +1.4]	67.7	67.9	−0.2 [−1.4, +0.9]

Table 21 Accuracy of the jointly trained LEV-2.6B and of a model trained for each depth alone (Single), with the paired difference in points. On JevBench the single-depth models reach 60.6%, 72.3%, and 77.9%, against 63.2%, 71.0%, and 76.2% for joint training.

Training only the last step. The model that runs four steps and is trained only at the last leaves its shallow readouts unusable (Section 7.5). Its depth one answers 36.4% of LevBench correctly against 46.9% for joint training, so its stopper almost never stops there and on-demand depth computes 84% of the deepest pass instead of 76%. It is the single-depth model for depth four of Table 21.

G.4 Losses and Loop Steps

Table 22 lists the recipe variants of Section 7.5 on the pooled held-out panels. A Brier term with weight one, added to every step's loss, raises the LevBench accuracy of the deepest readout of LEV-2.6B by 1.3 points but not its Kev-Breadth accuracy, and has no effect on LEV-3B. Weights that grow with the step, $w_t \propto t$, move training toward the deep readouts, so depth one loses 2.2 points on LevBench while the deepest readout stays level. LEV-2.6B trained and read with six steps gains at most one point after the fourth step on LevBench and none on Kev-Breadth. Its deepest readout is its sixth step, and its depth four reaches 65.9%. LEV-3B trained and read with three steps gains 0.5 points from its third step on the pooled held-out panels. A second epoch of LEV-1.4B at half the peak learning rate, started from the first, stays within one point of the first epoch on JevBench at every depth.

Recipe	Depth 1	Deepest	On-demand	Cost	vs. frontier
LEV-2.6B					
default	50.5	65.3	64.6	0.76	+0.2 [−0.4, +0.8]
Brier term	51.2	65.9	65.2	0.78	+0.2 [−0.4, +0.7]
$w_t \propto t$	48.6	65.8	65.0	0.78	+0.3 [−0.2, +0.9]
last step only	38.4	65.1	64.4	0.82	+1.1 [+0.6, +1.6]
six steps	50.0	66.2	65.7	0.67	−0.2 [−0.7, +0.4]
LEV-3B					
default	56.2	62.1	61.3	0.85	+1.0 [+0.6, +1.3]
Brier term	56.4	61.6	61.0	0.85	+0.9 [+0.5, +1.2]
three steps	55.7	62.7	62.1	0.66	+0.2 [−0.3, +0.6]

Table 22 Recipe variants on the pooled held-out panels. Cost and the difference to the frontier are for on-demand depth, and cost is a share of each model's own deepest pass.